Aircraft Noise: design summary

Decisions and changes up to 22 October 2026, why, and what's left · DESE71003 Sensing & IoT coursework

> **Goal.** Two microphones hear an aircraft; a classifier trained on the octave-band levels decides it's an aircraft; the ADS-B receiver identifies which one (the aircraft whose sound would arrive at that moment); the Sonos announces it with its route. **Only aircraft that were actually heard are announced.** v1 announces every heard flight; v2 announces only unique ones, judged from how often each airline, type and route appears. Data is collected first; v1 is built once collection is complete.

1. Where the system stands (collection goes live tonight)

- **Two mic nodes** (ESP32 + INMP441, mains powered) turn every second of sound into nine numbers and send them, **encrypted**, over ESP-NOW long-range radio to a **bridge** ESP32 on the Pi's USB. No audio leaves a node.
- **The Pi** runs two services: `bridge_link` stores the levels; `adsb_logger` stores every aircraft position within 60 km plus one **pass** row per aircraft within 5 km (the ground truth the mics are checked against). Everything goes into one SQLite database. `analysis/summary.py` produces the time-series figures. The software flow is in Appendix A.
- **Fresh start:** the first 2.2 days of data are archived on the Mac, outside git, as the original database plus CSV copies (they open in Numbers or Excel) and a README explaining every column. The 27 h of raw bridge audio from the first trial were deleted, not archived. Code went from about 60 files to 27, with one README.

2. Recording efficiency: from audio to levels

Recording method	On the air	Stored per mic per day	Privacy
Raw audio, 16 kHz × 16 bit (first trial)	32,000 B/s	2.76 GB	Records speech
Raw audio as 8-bit μ-law (tried, to fit the CH340 USB chip)	16,000 B/s	1.38 GB	Records speech
Now: 1 s levels (LAeq, LAmax, 7 octave bands, flags)	**44 B/s**	**≈ 11.7 MB**	**Speech can't be recovered**

727× less radio data and about 235× less storage per mic than raw audio, with the analysis done where the sound is. Each node filters 16,000 samples a second (A-weighting and seven octave-band filters, about 20 small filter stages per sample), which uses only a few percent of the ESP32. Octave-band levels are what airport noise monitors record, so the data stays useful for telling aircraft from other sounds and for training a model. ADS-B adds about 50 MB a day; the Pi has 49 GB free.

3. Why ESP-NOW rather than the home Wi-Fi, and how it scales

	ESP-NOW long range + bridge (chosen)	Home Wi-Fi straight to the Pi
Reach	Long-range mode: up to ~2× the range of normal Wi-Fi in open air; doesn't depend on the router	Weak at the windows, where the mics sit outside
Independence	Its own link: router restarts, password changes or busy Wi-Fi don't affect it	Depends on the router, DHCP and credentials in the firmware
Security	AES-CCMP encryption; the bridge accepts only the listed nodes; the household network is never touched	WPA2 of the home network; nodes join that network
Modularity	Nodes, relay and Pi are separate parts with one simple packet format	Fewer boards, but nodes tied to the house network
Cost	One extra ESP32 (the bridge) and one USB cable	None

Scalability. Each node sends one 44-byte packet a second, which at the long-range bit rate (¼–½ Mbit/s) takes a few milliseconds: under 1% of the air time, so the radio channel could carry well over a hundred nodes. The tighter limits are: **encrypted peers** on the bridge (7 by default, 17 at most, 20 peers in total), so about 7 nodes as configured and 17 with one setting changed; and the **USB serial link** at 115,200 baud (about 11.5 kB/s, roughly 50 nodes at one ~210-byte JSON line each per second). Beyond that, add a second bridge. On the Pi each node adds about 11.7 MB a day and one database row a second, which SQLite handles easily. Adding a node means one line in `protocol.h`, one in the Pi's mic table, and a reflash.

4. Sound propagation model

An aircraft is treated as a point source. Its sound spreads over a sphere, losing 6 dB per doubling of distance, and the air absorbs a little more with every kilometre:

$$L_{A,max}(r) = L_1 - 20\log_{10}(r / 1\ \text{km}) - \alpha\,(r - 1\ \text{km})$$

r is the slant distance at the aircraft's closest point (from ADS-B), $\mathbf{L_1}$ the level at 1 km and **α** the air absorption in dB/km. Both are fitted per mic, by least squares, to the fixed-wing passes, after subtracting the background's energy (as in ISO 1996), and

only over the distances where the mic hears at least 90% of passes. Further out, only passes that happened to be louder get heard, which flattens the curve and hides the absorption. The sound arrives **r / 343 m/s** after the closest moment (3.2 s at 1.1 km), so that's where each pass's level is read. The model is used three ways:

- **Predict and emulate before collecting:** synthetic data was generated from the model with known values (α = 4.0 dB/km) for three random datasets and a full week. The analysis recovered the 1 km level to within 0.2 dB and α at 4.2–4.5 dB/km, which checked the whole pipeline end to end, and a week of both mics runs in 8 s.
- **Attribution:** the sound delay tells v1 which aircraft a detected sound belongs to.
- **Insight:** the fitted curve and its scatter show how far away planes are still audible at each mic, and which aircraft are louder than the model predicts (a feature for v2's "special aircraft").

Air absorption rises steeply with frequency (ISO 9613, 10 °C, 70% humidity: about 0.1, 0.4, 1.0, 1.9, 3.7, 9.7 and 33 dB/km in the 63 Hz to 4 kHz octaves), so a pass sounds more muffled the further away it is. The octave bands let that be tested directly: the high bands of a pass should fall faster with distance than the low ones. Simplifications: no Doppler, no ground reflection, and one source level per aircraft type. Distances use GPS altitude, which 847 of 850 nearby aircraft sent in the trial, converted from height above the WGS84 ellipsoid (what aircraft broadcast) to height above sea level by subtracting the geoid height, about 46 m in London, and the receiver's height (about 8 m above sea level). There's no barometric sensor: the few pressure-altitude passes are flagged, and could be corrected later from Heathrow's archived weather reports. Aircraft on the ground are excluded, and helicopters are kept out of the fit.

5. From mic to announcement: how the pieces fit

Stage	Driven by	What it does
Pass records	ADS-B	Every aircraft within 5 km, heard or not: the ground truth. Lets the report measure what the mics *missed* (detection rate by distance, altitude and window), and labels the training data.
Heard events (v1, built after collection)	The mics	The **classifier**, trained on the octave-band levels labelled from the pass records, decides each moment is an aircraft; the detection is matched to the pass whose sound would arrive then.
Flight info (to build)	Heard events only	Route by callsign from the free, offline **VRS standing data** (updated daily); aircraft type, owner and photo from **adsbdb** (free, cached). No FlightAware or feeding for now: nothing is looked up for flights that weren't heard.
Announcement (to build)	Heard events only	Spoken on the Sonos over whatever is playing (method already proven in `Sonos-Test`). v1: every heard flight, within quiet hours. v2: unique aircraft only.

About **460 aircraft a day** pass within 1.5 km, roughly **one every two minutes** in daytime, almost all on the Heathrow approach at about 3,250 ft. That gives plenty of training examples within a couple of weeks, and makes v2's unique-only filter essential for everyday use. British Airways alone made 454 of 1,044 close passes in the trial, while 25 airlines appeared only once.

6. Data, privacy and security

- **No audio is recorded or stored** anywhere: privacy by design, and the main ethics point. The first trial's audio was deleted.
- **The home location** lives only in `pi/location.json`, which git ignores. Published data will carry distances and bearings from the receiver, never coordinates, because aircraft positions around a receiver reveal where it is.
- **Radio:** AES-CCMP encrypted ESP-NOW. The keys are generated randomly by `flash.sh` and kept in a git-ignored `secrets.h`. The bridge rejects unknown boards and wrong packet versions.
- **The publishing API and dashboard** are deferred until the model is trained, for the deployment part of the report.

7. Robustness built in

- Nodes are mains powered and always listening; they **restart themselves** if the code hangs (watchdog) or the bridge stops acknowledging for 60 s. The bridge sends a heartbeat every minute.
- **Mic-fault flag** when a mic sends nothing but one value. A faulty INMP441 found during testing read pure zeros and crashed the Pi's USB controller; new mics are now tried on a phone charger first.
- Sequence numbers expose lost packets; `uptime` exposes restarts. The Pi logs a health line per mic every 10 minutes and keeps its logs across reboots. Services restart by themselves after a power cut. The ADS-B logger records its own coverage, so an outage can't pass for an empty sky.
- **Calibration** belongs to each mic and lives on the Pi (no reflash). It's done with both nodes in their weatherproof cases, side by side, with pink noise at three fixed levels: an exact A-vs-B offset, an approximate absolute offset from a phone meter, and a check on whether the cases colour any octave band.

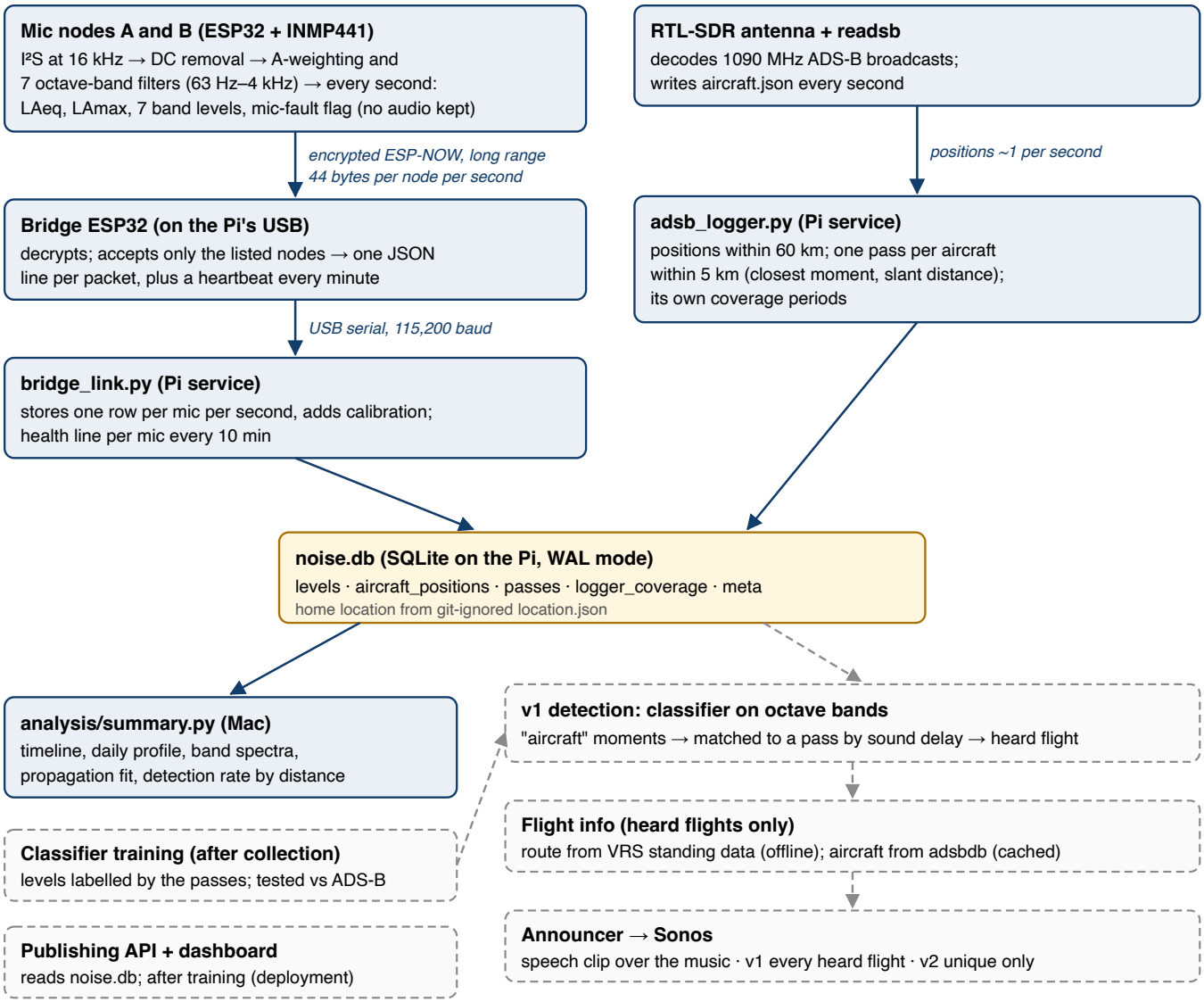
8. Build plan: collect, then v1, then v2

Collection first (about two weeks from 23 October): both mics and ADS-B, with nothing else to build meanwhile. **v1** is built once collection is complete, so the classifier trains on the whole dataset. A **classifier on the octave-band levels** (with LAeq and LAmax, over a short window around each second) decides "aircraft or not". Its labels come automatically from the pass records: *aircraft* when a close pass's sound was arriving, *no aircraft* when nothing was within 15 km for a minute either side, and ambiguous moments dropped. It's tested on held-out days against the ADS-B ground truth (precision and recall, by distance and by mic), with a simple level-threshold rule as the comparison baseline in the report. Each detection is matched to its pass by the sound delay, the route and aircraft are looked up, and every heard flight is announced. **v2's** only planned change so far: announce **unique aircraft only**, judged by how rarely the airline, type, route and registration appear in the pass history.

The full decision log is in Appendix B, and the remaining work, in order, is in Appendix C.

Appendix A. Software workflow

Solid boxes run from tonight. Dashed boxes are built after data collection (Appendix C).



Appendix B. Decision log

Decision	Why
Store levels, never audio	Privacy, and the module's ethics guidance on sensitive data; 727× less radio data and about 235× less storage. The first trial's audio (27 h) was deleted.
LAeq + LAmax + 7 octave bands, each second	The standard way aircraft noise is monitored; enough detail to tell aircraft from traffic and to train a model on; can't reproduce speech.
Mains-powered, always-on nodes	The nodes are wired to power, so sleep, heartbeats and battery code were removed in favour of robust, continuous data.
Keep the bridge and ESP-NOW (not Wi-Fi)	House Wi-Fi is weak where the mics sit, outside the windows; keeps the sensing network independent of the household network; modular.
ESP-NOW long-range mode	The data is tiny (44 B/s), so the slower bit rate costs nothing and buys range for the untested floor-3 bedroom.
Encrypt ESP-NOW	Security: only the listed nodes are accepted, and keys stay out of git.
Bridge has no mic	Its mic moved to a node. With nothing connected, its data pin floated and reported junk (70–87 dB(A)).
One firmware for both nodes; mics identified by MAC	No per-node builds. Names, locations and calibration live in one table on the Pi, so moving or recalibrating a mic needs no reflash.
Watchdogs, mic-fault flag, health log, persistent logs	Lessons from testing: a dead mic, a crashed USB controller and two power cuts, which were hard to diagnose without logs.
Serial at 115,200 baud	The CH340 USB chip dropped bytes above about 40 kB/s while audio was being sent; levels need under 1 kB/s.
Remove the 50 Hz high-pass; drop the unweighted level	A-weighting and the band filters already reject sub-audible rumble; fewer stages, same information.
A-weighting (dB(A))	Matches what people hear and what a phone meter shows; unweighted levels were 10–15 dB higher in rumbly rooms.
ADS-B antenna stays indoors	The trial showed complete coverage to 60 km in all 12 directions, with positions every ~1 s for nearby aircraft; outdoors would only add far-away range.
Pass records (ADS-B) and heard events (mics)	Passes give the ground truth needed to measure misses and to label training data; mic events drive the lookups and announcements.
Routes from VRS standing data; aircraft from adsbdb	Free, tested (BAW906 → Heathrow–Frankfurt), and VRS works offline. FlightAware's free allowance ($5–10/month) wouldn't cover ~460 passes a day; no feeding for now.
v1 uses the classifier, built after collection	The classifier on octave bands is the learning goal and the core of v1; building it once collection is complete means it trains and is tested on the full dataset. A simple level-threshold rule is kept only as a comparison baseline.
v2: announce unique aircraft only	About one pass every two minutes in daytime makes "announce everything" too much for daily use; uniqueness is learned from the pass history. The only v2 change planned so far.
Receiver height in slant distance	The Pi and mic A are one floor up (about 8 m above sea level, set in `location.json`); mic B is one floor higher, which changes distances to aircraft by under 0.5%.
GPS altitude first; pressure altitude flagged	847 of the 850 aircraft within 5 km in the trial sent GPS altitude. Pressure altitude isn't corrected for the day's pressure (about ±30 ft per hPa), so passes record which was used. Pressure can be corrected afterwards from Heathrow's archived METAR weather reports if ever needed.
GPS altitude converted to sea level	Aircraft broadcast GPS height above the WGS84 ellipsoid, about 46 m (150 ft) above London's sea level. Uncorrected, every slant distance was ~5% too long overhead (~0.4 dB in the fit, ~0.13 s of sound delay), and a helicopter on a pad read ~150 ft, slipping past the ground filter. Raw values are kept, so old rows can be corrected too.
No barometric sensor (for now)	With GPS altitude on nearly every nearby aircraft it adds little; the classifier never uses altitude, and stored raw altitudes allow later correction.
Aircraft on the ground aren't passes	Taxiing aircraft and helicopters on the nearby heliport pad would pollute the labels and the fit. Flagged on-ground, or below 100 ft and slower than 30 kt, is excluded; hovering and landing helicopters stay in, and their ADS-B category keeps them out of the fixed-wing propagation fit.
Database writes batched; a busy database never crashes a service	The mic-level writer held SQLite's write lock almost constantly, so the ADS-B logger crashed on "database is locked" and restarted: the cause of the trial's 16-second gaps. Levels are now written in one short transaction per second, and both services skip a beat instead of crashing.

Analysis built for weeks of data	Sorted arrays and binary search: a week of both mics (1.2 M rows, 3,668 passes) runs in 8 s with 460 MB, where the first version would have needed tens of GB.
Strict "quiet" baseline	No aircraft within 15 km slant for a minute either side, and the ADS-B logger running: planes at 3–5 km no longer count as quiet.
Fit only where detection is complete	Near the edge of hearing only the louder passes get heard, which biased the absorption low. Fitting only where the mic hears at least 90% of passes recovered the emulated values on three random datasets.
Fresh database; first run archived	A clean dataset with one schema; the 2.2-day trial is kept on the Mac (outside git) as the original database plus readable CSV files and a README.
Location in a git-ignored file	Code and data can be published on GitHub without revealing the home.
Publishing API and dashboard later	They belong to the deployment section, once the model exists.
Simplify: ~60 files → 27, one README	Every one-off test tool, unused option and extra board type removed; easier to reason about and to report.

Appendix C. What's left to do, in order

Tonight: start collecting

1. **Flash node A, then node B.** Plug each into the Pi in turn (or the Mac) and run `esp32/flash.sh node pi` (or `esp32/flash.sh node` on the Mac). The bridge is already flashed. All three boards must use the same encryption keys, which are in `esp32/common/include/secrets.h` on the Mac: don't delete that file.
2. **Switch the Pi to the new services:** `ssh -t sebpi@sebpi sudo bash aircraft-noise/setup/install.sh` (needs your password).
3. **Start:** clear any old-format database the old services created, then remove the pause file (`ssh sebpi@sebpi rm aircraft-noise-data/PAUSED`).
4. **Check after 10 minutes:** the health log should show both mics with few lost packets and no mic faults (`journalctl -u bridge-link`).

This week

5. **Calibrate:** both nodes in their cases, side by side, pink noise at about 50, 65 and 80 dB(A), a minute each, phone meter alongside. Put the offsets in the mic table and redeploy.
6. **Install the nodes** at their windows, then check the radio after a day (signal strength and lost packets). The floor-3 bedroom is the one to watch.
7. **Email Dr Boyle** briefly about sensing in a shared home (no audio recorded), as the brief asks for anything involving people's data.
8. **Keep a note of generative AI use** (Claude Code for code and design discussion), for the required notice.

During collection (about two weeks, to around 6 November)

9. **Let it run.** Glance at the health log every few days, and after the first week run `analysis/summary.py` to check the data looks right: levels, close passes, lost packets, mic faults. Fix anything odd early. There's nothing else to build during collection.

After collection: v1

10. **Train the classifier:** features from the octave bands, LAeq and LAmax over a short window around each second; labels from the passes (aircraft, no aircraft, ambiguous dropped). Hold back some days for testing, and report precision and recall against ADS-B, by distance and by mic, alongside the simple threshold baseline.
11. **Match detections to flights:** each detected aircraft sound goes to the pass whose sound would arrive then.
12. **Flight info:** download the VRS route files weekly; route by callsign for each heard flight; aircraft type and owner from adsbdb, cached per aircraft.
13. **Announcer:** reuse the Sonos-Test clip method; generate the spoken sentence; quiet hours and a minimum gap. Then run v1 live: classifier, match, look up, announce.

v2

14. **Unique aircraft only:** score each heard flight by how rarely its airline, type, route and registration appear in the pass history, and announce only the unique ones.

Deployment and report (deadline 10 December, 4 pm)

15. **Publishing API and dashboard** on the Pi (read-only JSON and a web page: live levels, recent heard flights).
16. **Analysis:** `analysis/summary.py` on the full dataset, plus classifier performance, announcement delay, packet loss and uptime, data volume, scalability, security and privacy.
17. **Publish the code and data on GitHub:** export the dataset with distances and bearings only (no coordinates).
18. **Write the report** (cover page and 6 pages, IEEE or ACM two-column) and record the 5-minute video. Include the AI-use notice.

Suggested dates: collecting 23 Oct to about 6 Nov · v1 (classifier, matching, flight info, announcer) by 20 Nov · v2 by 25 Nov · API, dashboard and analysis by 30 Nov · report and video by 7 Dec.